

Computation Beyond Data

Constraints, Reachability, and Memory
as Foundations of Complex Systems

Flyxion

Abstract

Much of modern computer science rests on an assumption so pervasive it is rarely examined: data comes first, and algorithms operate upon it. This essay proposes a partial inversion of that perspective. Rather than treating objects, data, or states as the fundamental primitives, the focus shifts to the constraints, admissible trajectories, and reachability spaces that make certain states accessible and others impossible. This shift is not merely philosophical. It has direct consequences for how we design AI systems, model computational memory, understand distributed architectures, and theorize cognition. The essay presents five interconnected frameworks — CLIO, RSVP, MEM|8, HYDRA, and TARTAN — as instances of a unified theoretical program: the claim that constraint precedes content, that reachability is more fundamental than distance, and that the most productive question about any complex system is not *what exists?* but *what remains possible?*

Overview of the Frameworks

The five frameworks discussed in this essay operate at different levels of abstraction. CLIO addresses inference under partial observation: how intelligent systems reconstruct world-states from incomplete projections. RSVP addresses system dynamics and structural relaxation: how complex systems evolve toward configurations that minimize internal tension. MEM|8 addresses memory and persistence through event trajectories: treating stored state not as a value but as the cumulative record of decisions that produced it. HYDRA addresses modular cognitive architecture: how robust intelligent behavior emerges from the coordination of specialized components rather than from a single monolithic mechanism. TARTAN addresses structure-preserving decomposition: how complex systems can be transformed while guaranteeing that specified properties survive the transformation. Although each framework can be studied independently, they are presented here as components of a broader admissibility-centered view of computation — one in which the structure of what is possible is at least as important as the structure of what currently exists.

Contents

1	The Hidden Assumption	4
2	The Central Principle: Constraint Before Content	4
2.1	The Chess Game	4
2.2	The Constraint Principle	5
2.3	Constraints as Information	5
3	CLIO: Intelligence as Navigation Through Projection Spaces	6
3.1	The Problem of Partial Observation	6
3.2	Uncertainty as Structure	6
3.3	Intelligence as Efficient Ambiguity Reduction	6
3.4	Projection Failure and Paradox	7
4	RSVP: Complex Systems as Processes of Relaxation	7
4.1	From Cosmology to Computation	7
4.2	The Potential Function and Gradient Descent	8
4.3	Attractors and Stable Configurations	8
4.4	Entropy, Information, and Geometry	8
5	MEM 8: Memory as Trajectory Rather Than State	9
5.1	The Conventional Model of Memory	9
5.2	MEM 8: Memory as a Record of Collapses	9
5.3	Connections to Existing Technologies	10
5.4	Wave Memory and Persistence	10
6	HYDRA: Cognition as Modular Coordination	11
6.1	The Problem of Cognitive Architecture	11
6.2	HYDRA: Multiple Specialized Modules	11
6.3	Connections to Modern AI Architectures	12
7	TARTAN: Structure-Preserving Decomposition	12
7.1	The Problem of Context Loss	12
7.2	Structure-Preserving Decomposition	12
7.3	TARTAN and Admissibility	13
8	Reachability Rather Than Distance	13
8.1	Shifting the Question	13
8.2	Reachability Volume	14
8.3	Freedom as Reachability Volume	14

9	Artificial Intelligence as a Simulation Engine	15
9.1	Perception as Inference	15
9.2	Memory as Reconstruction	15
9.3	Reasoning as Simulation	15
9.4	Apparent Intentionality and the Attention Loop	16
10	The Unified Core: Admissibility	16
10.1	The Central Concept	16
10.2	Admissibility as Geometry	17
10.3	The Unifying Question	17
11	Implications for Practice	17
11.1	System Design	17
11.2	Evaluating AI Systems	18
11.3	Memory Engineering	18

The Hidden Assumption

Every student of computer science learns early to think in terms of data and operations. There is an array; we apply a sorting algorithm. There is a graph; we run a breadth-first search. There is a labeled dataset; we train a classifier. The general structure is always the same: the world supplies objects, and we process them.

This assumption is so fundamental that it is rarely examined. It surfaces in basic terminology — variable, value, state, datum — and silently organizes the way we think about nearly everything in the field.

There is, however, an alternative orientation. Rather than beginning with objects and asking what we can do with them, we can begin with constraints and ask which objects they make possible. Rather than beginning with states and asking how we transition between them, we can begin with state spaces and ask which regions are reachable from where we currently are.

This inversion produces different questions, different models, and, the essay argues, a deeper understanding of several of the hardest problems in modern computing: the generalization problem in machine learning, the consistency problem in distributed systems, the persistence problem in computational memory, and the emergence problem in cognitive systems.

The goal of this essay is to present a set of theoretical frameworks that start from this inversion and trace its consequences across different domains. The frameworks are independent enough to be studied separately, but they share a conceptual core that makes them parts of a unified theoretical program.

The Central Principle: Constraint Before Content

The Chess Game

Consider a game of chess. The pieces matter, but the rules defining the possible moves matter more. Without rules, there is no game — only a collection of wooden or plastic objects on a checkered surface. The game emerges from the constraints, not from the components.

This is not trivial. It implies that the identity of a piece — what it *is*, functionally — is entirely determined by the constraints governing its behavior. A king that could move like a queen would no longer be a king. Changing the rule changes the object, even when the physical piece remains the same.

The same logic appears throughout computation.

In compilers, a formal grammar defines the space of syntactically valid programs. A program is not merely a sequence of characters; it is a sequence satisfying the constraints imposed by the grammar. The grammar precedes and constitutes the program.

In relational databases, the schema defines which records can exist. A database is not merely a collection of data; it is a collection of data satisfying a set of integrity constraints. Violating those constraints is not simply a technical error; it is a contradiction with the very definition of what the database represents.

In neural networks, the architecture defines the space of functions the network can learn. Before seeing any training example, the architecture already determines which representations are possible and which are not. The architectural constraint precedes the data.

In operating systems, the permission model defines which actions each process may execute. Security is not an add-on appended after the system is built; it is the constraint structure within which the system operates.

The Constraint Principle

Constraint Principle. The constraints that define a system are not implementation details added to prevent errors. They are the primary source of the system's organization — the structure that determines which states exist, which transitions are possible, and which objectives can be pursued.

This perspective has an immediate consequence: to understand a complex system, the first question is not *what elements does it contain?* but *what constraints organize it?*

Constraints as Information

There is a deep connection between constraints and information that deserves to be made explicit. Shannon defined information as the quantity that reduces uncertainty. A message is informative to the degree that it eliminates possible alternatives.

Constraints do exactly this. A constraint eliminates possible states. The more constrained a system, the smaller its reachable state space — and therefore the more information is implicitly encoded in its constraint structure.

This observation has practical implications. Every design decision that reduces the space of possible states also encodes information about the system's purpose. A system without constraints is a system without purpose — and a system without purpose is computationally inexpressive.

Proposition 1 (Constraint-Information Equivalence). *Let S be the complete state space of a system and C the subset of admissible states under a constraint set R . The information encoded by R is proportional to $\log |S| - \log |C|$: the difference between the unconstrained space and the constrained space. Stronger constraints encode more information.*

CLIO: Intelligence as Navigation Through Projection Spaces

The Problem of Partial Observation

Intelligent systems rarely observe reality in full. They receive partial projections of the world: pixels instead of objects, tokens instead of meanings, sensory signals instead of physical states.

Formally, let X be the complete world-state space and M the observation space available to the system. There exists a projection function

$$\pi : X \rightarrow M$$

mapping world-states to observations. In general, π is not injective: multiple world-states can produce the same observation.

The central problem of intelligence, from this perspective, is the inversion of this projection: given $m \in M$, what is the set

$$\pi^{-1}(m) = \{x \in X \mid \pi(x) = m\}$$

of world-states consistent with the observation?

Uncertainty as Structure

The preimage $\pi^{-1}(m)$ represents the system's structural uncertainty: the ambiguity that is mathematically inherent in the projection, independent of any processing limitation. A system observing m cannot distinguish between any of the states in $\pi^{-1}(m)$ on the basis of that observation alone.

Several important consequences follow.

For machine learning: the generalization problem is fundamentally the problem of inverting a projection. A model trained on examples has seen only a sample of the world-state space. Generalization is the capacity to reconstruct the structure of the state space from those samples.

For perceptual systems: perceptual ambiguity is not a bug that can be eliminated with more data or better hardware. It is a structural property of the relationship between sensors and world.

For reasoning under uncertainty: Bayesian uncertainty is a way of representing $\pi^{-1}(m)$ through probability distributions over world-states.

Intelligence as Efficient Ambiguity Reduction

CLIO defines intelligence operationally as the capacity to reduce the space $\pi^{-1}(m)$ efficiently, using additional constraints derived from context, memory, and reasoning.

Definition 1 (CLIO Intelligence). *A system is intelligent to the degree that, given $m \in M$, it can identify a subset $\mathcal{A}(m) \subseteq \pi^{-1}(m)$ of plausible states such that $|\mathcal{A}(m)| \ll |\pi^{-1}(m)|$, using constraints derived from context and memory, without direct access to X .*

This definition unifies several concepts that ordinarily appear separately in AI.

Reasoning is the application of logical constraints to reduce $\pi^{-1}(m)$.

Learning is the acquisition of constraints that make reduction more efficient in new contexts.

Memory is the storage of constraints derived from prior experience.

Generalization is the transfer of constraints to unobserved contexts.

Projection Failure and Paradox

An important consequence of the CLIO framework is a natural account of certain classes of paradox and cognitive illusion.

Definition 2 (Projection Failure). *A projection failure occurs when a system treats the projection m as if it were the complete state x — when it confuses the map with the territory.*

Many systematic errors in human reasoning and in AI systems are instances of projection failure. Cognitive biases arise when the system applies heuristics that work well for the majority of states in $\pi^{-1}(m)$ but fail for less typical states. Hallucinations in large language models arise when the model treats its internal representation as the actual state of the world.

Semantic paradoxes arise when a description m is treated as a direct reference to a state x , when in fact $\pi^{-1}(m)$ contains multiple states with incompatible properties. Restoring the context — specifying which state in $\pi^{-1}(m)$ is intended — frequently dissolves the apparent paradox.

RSVP: Complex Systems as Processes of Relaxation

From Cosmology to Computation

RSVP was originally developed as an alternative cosmological framework, but its mathematical structure admits a direct computational interpretation that is independent of its original motivation.

The central claim is that complex systems evolve toward states that reduce internal structural tension. Rather than modeling a system as a static collection of objects in fixed states, RSVP describes it as a continuous dynamic process of constraint reorganization.

The Potential Function and Gradient Descent

Formally, let $x(t)$ be the state of a system at time t and $V(x, t)$ a potential function representing internal structural tensions — violated constraints, unresolved gradients, inconsistencies among subsystems. The system evolves approximately by gradient descent:

$$\frac{dx}{dt} = -\nabla_x V(x, t) + \xi(t),$$

where $\xi(t)$ represents external perturbations.

Computer scientists will recognize this structure immediately: it is the same as stochastic gradient descent in machine learning, variational optimization in generative models, and diffusion equations in physical systems.

The difference in perspective is that RSVP treats the function V not as an objective chosen by the designer but as an emergent property of the system’s constraint structure. The “optimization” is internal and requires no explicit external objective.

Attractors and Stable Configurations

An attractor is a region of state space toward which the system converges naturally. In the RSVP perspective, attractors correspond to configurations that locally minimize structural tension.

Proposition 2 (Attractors as Tension Minima). *The stable configurations of a complex system correspond to local minima of the tension function $V(x, t)$. A perturbation destabilizes the system by increasing V beyond a critical threshold, causing the local minimum to lose stability — formally, when the Hessian $\nabla_x^2 V(x^*, t)$ loses positive definiteness.*

This framework has direct applications for understanding:

Neural network training: the training process is a search for minima of a loss function. The loss landscape contains multiple local minima, and different hyperparameter configurations correspond to different tension functions.

Distributed systems: eventual consistency in distributed systems can be understood as convergence toward a global attractor from locally inconsistent states.

Language models: text generation can be viewed as navigation through a linguistic state space with an implicit tension function defined by the training data.

Entropy, Information, and Geometry

One of RSVP’s richest conceptual contributions is the reinterpretation of entropy and information as geometric properties of system dynamics.

Rather than treating entropy as a scalar quantity associated with a probability distribution, RSVP treats it as a field density: a quantity varying in space and time that encodes local structural tensions.

This perspective connects naturally to:

Variational inference: which minimizes a divergence between an approximate and a target distribution — interpretable as tension minimization in a space of distributions.

Normalizing flows: which learn transformations mapping simple distributions to complex ones while preserving volume structure — a form of navigation through configuration spaces.

Information geometry: which studies the curvature of the space of probability distributions — analogous to the curvature of configuration space in RSVP.

MEM|8: Memory as Trajectory Rather Than State

The Conventional Model of Memory

The conventional model of computational memory is simple and efficient: memory is a map from addresses to values. Reading memory retrieves a value. Writing memory replaces one value with another. The history of writes is not preserved; only the current state matters.

This model is enormously powerful for most applications, but it has structural limitations that become visible in certain contexts.

In distributed systems, two processes may have written different values to the “same” address. The current state alone is insufficient to resolve the conflict; the ordering of writes must be known.

In version control systems such as Git, the current state of a repository is insufficient to understand how it arrived there. The commit history is an essential part of the representation.

In audit databases, it is necessary to preserve not only the current state but the complete sequence of modifications.

In cognition, identity is not the current state of an organism but the continuity of its trajectory through time.

MEM|8: Memory as a Record of Collapses

MEM|8 inverts the conventional model. Rather than treating memory as storage of states, it treats memory as a record of decisions — more precisely, as a record of successive collapses of possibility spaces.

Definition 3 (MEM|8 Memory). *A MEM|8 memory is a sequence*

$$\mu = (e_1, e_2, \dots, e_n)$$

of events, where each event e_i represents a decision that collapsed a possibility space $\mathcal{P}_{i-1}$ into

a reduced space $\mathcal{P}_i \subseteq \mathcal{P}_{i-1}$. The current state is

$$s_n = \bigcap_{i=1}^n \mathcal{C}(e_i),$$

where $\mathcal{C}(e_i)$ is the set of states consistent with event e_i .

This formulation has several consequences.

Identity is continuity: the identity of a system is not its current state but the trajectory that produced it. Two systems in the same state may have different identities if they arrived there by different trajectories.

Persistence is dynamic: the persistence of an entity is not a static state but an active process of maintaining trajectory continuity.

Retrieval is reconstruction: recovering a memory is not accessing a stored value but reconstructing a state from the sequence of events that produced it.

Connections to Existing Technologies

MEM | 8 has explicit connections to several established paradigms in software engineering.

Event Sourcing: an architectural pattern in which system state is derived from an immutable sequence of events. The current state is a projection of the event log, not an independently stored value.

CRDTs (Conflict-free Replicated Data Types): data structures that guarantee eventual consistency in distributed systems by defining merge operations that commute and are idempotent — analogous to systems where the ordering of events does not alter the final state.

Blockchains: immutable distributed ledgers in which the current state is completely determined by the transaction history. The immutability of the history is the integrity guarantee.

Temporal databases: databases that preserve the complete history of modifications, enabling queries over past states.

Version control systems: such as Git, where the identity of a repository is its commit tree rather than the current state of its files.

Wave Memory and Persistence

An extension of MEM | 8 treats memory not as a discrete sequence of events but as a continuous field of influences — a wave memory in which past events radiate influence that decays over time but never entirely disappears.

This formulation is relevant to:

Recurrent neural networks: where the hidden state encodes a form of short-term memory that continuously influences current processing.

Attention mechanisms: which allow language models to access information from prior context with weights that depend on relevance.

Episodic memory in AI: systems that store complete experiences and retrieve them by similarity with the current context.

HYDRA: Cognition as Modular Coordination

The Problem of Cognitive Architecture

How should an intelligent system be organized? The simplest answer is a monolithic system: a single model that receives inputs, processes them internally, and produces outputs. This architecture is mathematically elegant and computationally tractable.

It has limitations, however. Monolithic systems are difficult to inspect, difficult to debug, difficult to extend, and difficult to adapt to domains substantially different from their training domain. Beyond a certain scale, they become opaque black boxes whose behavior is difficult to predict or guarantee.

HYDRA: Multiple Specialized Modules

HYDRA takes as its central hypothesis that robust cognition emerges from the coordination of specialized modules rather than from a single monolithic mechanism. The modules are:

Perception: transforms sensory inputs into internal representations. This is not a direct data pass-through; it is an active projection that emphasizes certain dimensions and suppresses others.

Memory: maintains representations of past states and makes them available to other modules. In the MEM | 8 framework, this is the trajectory record.

Planning: projects possible futures from the current state and available constraints. This is fundamentally navigation through the reachable state space.

Coordination: ensures that modules operate coherently as a system. This module manages interactions and resolves conflicts among the outputs of the other modules.

Monitoring: evaluates system-level performance and detects when other modules are failing or operating outside their competence domains.

Integration: produces coherent responses by combining the contributions of the other modules.

Proposition 3 (Cognitive Emergence). *In HYDRA systems, complex cognitive behaviors emerge from the coordination of simple modules without any individual module possessing*

those behaviors. The coordination is the source of the complexity, not the sophistication of each module individually.

Connections to Modern AI Architectures

The central hypothesis of HYDRA — that cognition emerges from modular coordination — has parallels in several modern AI architectures.

Mixture of Experts (MoE): models in which different “experts” are activated for different inputs, with a routing mechanism that determines which expert to use.

Multi-agent systems: architectures in which multiple agents cooperate or compete to solve collective problems.

Multi-head attention: where different attention heads capture different types of dependencies in the input.

Neuro-symbolic systems: which combine neural networks with symbolic reasoning, typically through modules specialized for each paradigm.

HYDRA’s distinguishing feature is that it makes coordination explicit and tractable, rather than allowing it to emerge implicitly from training.

TARTAN: Structure-Preserving Decomposition

The Problem of Context Loss

A central problem in both distributed systems and AI is the loss of context during transformation. When information is divided, summarized, compressed, or transmitted between components, part of the original meaning disappears.

Formally, this is information loss in projections:

$$H(X) - H(\pi(X)) > 0,$$

where H is Shannon entropy. Any projection that is not bijective loses information.

The goal is not to eliminate this loss — which is frequently necessary for scalability and efficiency — but to control it: to ensure that the information most important to the system’s purpose is preserved.

Structure-Preserving Decomposition

TARTAN investigates how to decompose complex systems in ways that guarantee specified properties survive the transformation.

Definition 4 (TARTAN Decomposition). *A decomposition D of a system S is a TARTAN decomposition for a property set P if, for every property $p \in P$:*

$$p(S) = p(D(S)),$$

that is, the property is identical in the original system and in the decomposition.

The properties a system needs to preserve depend on context.

In distributed processing, the property may be eventual consistency — guaranteeing that all replicas converge to the same state.

In explainable AI (XAI), the property may be explanation fidelity — guaranteeing that the account of the model’s behavior accurately reflects what the model actually computes.

In semantic compression, the property may be preservation of inference relations — guaranteeing that what is inferable from the compressed version is exactly what would be inferable from the full version.

In multi-agent systems, the property may be coordination — guaranteeing that agents receiving different parts of a task produce results that combine coherently.

TARTAN and Admissibility

There is a deep connection between TARTAN and the concept of admissibility that runs through all the frameworks presented here.

A TARTAN decomposition for a property set P is, in essence, a way of ensuring that transformations applied to the system remain within the space of admissible transformations — the space of transformations that preserve the relevant properties.

This connects TARTAN to:

Category theory: which studies structure-preserving transformations (functors, natural transformations).

Homomorphisms: in algebra, transformations that preserve operations.

Bisimulations: in concurrency theory, equivalence relations between systems that preserve observable behavior.

Reachability Rather Than Distance

Shifting the Question

One of the most original conceptual contributions of the theoretical program presented here is the replacement of the question *how far away is something?* with the question *how difficult is it to reach?*

In computation, this shift is fundamental.

A file may be stored on the same physical disk as a process and be completely inaccessible to that process, whether through permissions, metadata corruption, or file-system failure.

A datum may exist in a distributed database and be unrecoverable due to network failures or partitions.

A state may be theoretically reachable in a control system but computationally infeasible given available processing power.

A solution may exist for an optimization problem but lie in a region of the search space that available algorithms cannot efficiently explore.

In all of these cases, the physical or logical distance between the current state and the desired state is irrelevant. What matters is *reachability*: the existence of an admissible path between the two states.

Reachability Volume

Definition 5 (Reachability Volume). *For a system in state x , the reachability set $R(x)$ is the set of all states reachable from x by sequences of admissible transitions. The reachability volume is*

$$\Omega(x) = \text{Vol}(R(x)).$$

The reachability volume is a measure of the system's *freedom* — how broad the space of possible futures is from the current state. Systems with high reachability volume have many options; systems with low volume are essentially determined.

This metric appears, under different names, across several areas.

Computational complexity theory: the class of problems an algorithm can solve efficiently is its computational reachability space.

Reachability in control systems: the set of states a control system can reach from a given initial state is its reachability set.

State graphs in model checking: verifying properties of concurrent systems involves exploring the graph of reachable states.

Search spaces in AI planning: planning algorithms navigate state graphs seeking sequences of actions that reach a goal state.

Freedom as Reachability Volume

There is an important connection between reachability and freedom that extends beyond computation.

Proposition 4 (Reachability Principle — Computational Version). *The effective freedom of an agent is proportional not to the resources it possesses but to the volume of future states reachable from its current state:*

$$\text{Freedom}(x) \propto \Omega(x).$$

This proposition has immediate practical implications for system design. A software system that progressively reduces the reachable state space for its users — by accumulating internal state, creating data dependencies, or restricting configuration

options — reduces users' effective freedom, independently of any enrichment of functionality. An AI system that converges toward an increasingly narrow output distribution as it processes more context is reducing its own reachability volume — potentially irreversibly.

Artificial Intelligence as a Simulation Engine

Perception as Inference

The hypothesis that intelligent systems function as simulation engines — constructing internal models of the world that enable them to project possible futures — has deep roots in cognitive science and connects directly to the frameworks presented here.

Within the CLIO framework, perception is not a direct data pass-through from environment to system. It is an inference process: given the sensory signal $m = \pi(x)$, the system constructs an estimate of the world-state x that minimizes structural tension between the observation and the expectations derived from the internal model.

This is mathematically equivalent to variational inference in generative models: minimizing a divergence between a posterior $p(x | m)$ and a variational approximation $q(x)$.

Memory as Reconstruction

Within MEM|8, memory is not retrieval of a stored state. It is reconstruction of a state from a sequence of events.

This has an important consequence: memories are necessarily incomplete and potentially inconsistent. When reconstituting a past state, the system interpolates over regions of the state space that were never directly observed.

This perspective is consistent with what is known about human memory: memories are active reconstructions, not passive retrievals. The central nervous system does not store experiences as files; it reconstructs experiences from fragments, context, and expectations.

For AI systems, the relevant question is not *how many* memories a system can store, but *how faithfully* it can reconstitute past states from compressed representations.

Reasoning as Simulation

Within the RSVP perspective, reasoning is the evolution of a system's internal state toward configurations that minimize structural tension. Solving a problem is finding a state that simultaneously satisfies a set of constraints.

This connects reasoning to:

Constraint satisfaction (CSP): finding variable assignments that satisfy a constraint set.

Constraint programming: a programming paradigm in which solutions are found through constraint propagation and satisfaction.

Inference in probabilistic models: finding distributions that minimize divergences with respect to observations and priors.

Apparent Intentionality and the Attention Loop

The most speculative hypothesis — but also the most interesting — is that consciousness, or something functionally equivalent to it in artificial systems, emerges from the continuous process of projection and correction: the system projects a model of the world, receives feedback about discrepancies between model and world, and iteratively corrects the model.

This process is not merely computation over data. It is computation over the system's own representation of itself and the world it inhabits — a form of self-reference that may be the basis of behaviors that appear, from the outside, to be genuinely intentional.

Systems that continuously update their internal models based on discrepancies between predictions and observations — that maintain, in some sense, an attention loop over their own world-representation — exhibit a form of emergent apparent intentionality that does not require consciousness in the strong philosophical sense but is functionally indistinguishable from it to external observers.

The Unified Core: Admissibility

The Central Concept

Across the frameworks presented here, one concept recurs under different names: the admissible state space, the reachability set, the constraints organizing the system, the subset of consistent projections. This concept unifies under the notion of *admissibility*: the structure that defines what is possible within a system.

Definition 6 (Admissibility Space). *For a system with state space S , the admissibility space $\mathcal{A} \subseteq S$ is the subset of states satisfying all active constraints. A transition $s \rightarrow s'$ is admissible if $s' \in \mathcal{A}$ and an admissible path from s to s' exists.*

Admissibility as Geometry

The richest perspective treats admissibility geometrically. The admissibility space $\mathcal{A}$ is a manifold (or sub-manifold) in the state space $\mathcal{S}$, with its own geometry induced by the constraints.

Operations on the system — state transitions, learning, inference, reasoning — are movements over this manifold. The curvature of the manifold encodes interactions among constraints. Regions of high curvature correspond to strongly interacting constraints; regions of low curvature correspond to independent constraints.

This perspective connects the frameworks presented here to:

Information geometry: which studies the curvature of the space of probability distributions.

Riemannian geometry in machine learning: which uses metrics induced by data structure to guide optimization.

Data manifolds: the hypothesis that high-dimensional data lies on low-dimensional manifolds embedded in the ambient space.

The Unifying Question

All five frameworks can be viewed as instances of a single question:

The Fundamental Question. Given a system in a current state x , with admissibility space $\mathcal{A}$, what is the set $R(x) \cap \mathcal{A}$ of admissible reachable futures? And how does this set change when the system's constraints change?

CLIO answers this question at the level of inference: the set of world-states consistent with an observation.

RSVP answers it at the level of dynamics: the attractors toward which the system converges naturally.

MEM|8 answers it at the level of memory: the states that can be reconstituted from a sequence of events.

HYDRA answers it at the level of architecture: the behaviors that emerge from modular coordination.

TARTAN answers it at the level of transformation: the properties preserved across decompositions.

Implications for Practice

System Design

The constraint-first perspective has concrete implications for software system design.

Specification before implementation: define the admissibility space before implementing the system. This is not simply writing tests; it is formally specifying the invariant properties the system must preserve.

Constraints as documentation: a system's constraints are more informative than its implementation. A constraint states what the system *must* do; an implementation states what it *does* under a specific set of conditions.

Admissibility monitoring: robust systems continuously monitor whether they are operating within their admissibility space and take corrective action when violations are detected.

Evaluating AI Systems

The most important question about an AI system is not its accuracy on a specific benchmark. It is the shape and size of its admissibility space: which inputs the system handles well, which it handles poorly, and where the boundary lies between those regimes.

A system with high in-distribution accuracy that degrades catastrophically on out-of-distribution inputs has a small admissibility space relative to the full input space. This is the generalization problem, reformulated geometrically.

Memory Engineering

MEM|8 suggests an approach to memory engineering that goes beyond index design and data structures.

Memory as an event log: design systems in which state is derived from an immutable event log rather than stored directly.

Consistency as invariant: treat consistency among replicas as an invariant the system must preserve, not as an objective to be achieved by synchronization protocols.

Identity as trajectory: design entity identities in distributed systems around trajectories rather than current states.

Conclusion

The frameworks presented in this essay — CLIO, RSVP, MEM|8, HYDRA, and TARTAN — appear at first glance to be disparate. One addresses inference, another dynamics, another memory, another architecture, another decomposition. Their domains of application range from cosmology to databases.

They share, however, a conceptual core that unifies them.

Reality, cognition, and computation can be understood as processes of navigation through spaces of possibility organized by admissibility structures. What exists at any instant is only the current realization of a much larger space of possibilities.

What matters, for understanding the system, is not only the current state but the constraint structure that defines which states are possible and which transitions are admissible.

This shift in perspective — from content to constraint, from state to trajectory, from distance to reachability — does not invalidate existing models and techniques. It reframes them in a way that makes certain questions easier to formulate and certain problems easier to attack.

Rather than asking only *what exists?*, this perspective introduces a different question:

What remains possible?

That question unifies physics, artificial intelligence, distributed systems, information theory, and cognition under a common language grounded in constraints, reachability, memory, and structural transformation.

References

- [1] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3–4), 379–423, 623–656.
- [2] Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory*, 2nd ed. Wiley.
- [3] Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos*. Westview Press.
- [4] Holland, J. H. (1995). *Hidden Order: How Adaptation Builds Complexity*. Addison-Wesley.
- [5] Meadows, D. H. (2008). *Thinking in Systems: A Primer*. Chelsea Green.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [7] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [8] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.
- [9] Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. *arXiv preprint arXiv:1312.6114*.
- [10] Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational Inference: A Review for Statisticians. *Journal of the American Statistical Association*, 112(518), 859–877.

- [11] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565.
- [12] Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-Free Replicated Data Types. *Symposium on Self-Stabilizing Systems*, 386–400.
- [13] Fowler, M. (2005). EventSourcing. martinfowler.com/eaDev/EventSourcing.html.
- [14] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- [15] Sipser, M. (2012). *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning.
- [16] Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.
- [17] Amari, S., & Nagaoka, H. (2000). *Methods of Information Geometry*. AMS & Oxford University Press.
- [18] Varela, F. J., Thompson, E., & Rosch, E. (1991). *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press.
- [19] Clark, A. (2015). *Surfing Uncertainty: Prediction, Action, and the Embodied Mind*. Oxford University Press.
- [20] Rossi, F., van Beek, P., & Walsh, T. (Eds.). (2006). *Handbook of Constraint Programming*. Elsevier.
- [21] Awodey, S. (2010). *Category Theory*, 2nd ed. Oxford University Press.
- [22] Schacter, D. L. (1996). *Searching for Memory: The Brain, the Mind, and the Past*. Basic Books.
- [23] Clarke, E. M., Grumberg, O., & Peled, D. A. (1999). *Model Checking*. MIT Press.